Claude Code Remote Control via WhatsApp — Technical Specification

Status: Implemented and in daily use

1. Purpose

Allow the project owner to send prompts to Claude Code — as text, voice notes, or file attachments — from a WhatsApp chat on their Android phone, and receive Claude Code's responses back as WhatsApp text messages. The system must run entirely on the owner's own Kubuntu desktop, using only a Claude Code Pro subscription (no Anthropic API billing), with no public IP address and no VPS.

2. Environment & Constraints

Item	Value
OS	Ubuntu/Kubuntu 26.04 LTS ("resolute"), KDE Plasma (Wayland)
CPU	Intel Core i7-12650H, 10 cores
RAM	30 GB
GPU	NVIDIA GeForce RTX 3060 Mobile, 6 GB VRAM, driver 595.84, CUDA 13.2
Claude Code auth	Pro subscription only — CLI must run under the interactive OAuth login session, **not** an API key
Network	No public IP, no VPS, no port-forwarding, no inbound tunnel
Messaging	Owner's everyday personal WhatsApp number, via Android WhatsApp app
Language	No hard preference; Python selected for orchestration (matches existing project scaffold), Node.js/TypeScript required for the WhatsApp layer (see §4.1)

Hard operational requirement: automatic suspend/sleep must be disabled on this machine. Screen lock / screensaver is fine (an active login session keeps running while locked), but if the machine sleeps, the WhatsApp connection drops and nothing can reach Claude Code until the machine is physically woken — there is no remote-wake path available under these constraints.

3. High-Level Architecture

```
flowchart LR
    subgraph Phone["Android — WhatsApp app"]
        Self["'Message Yourself' chat"]
    end

    subgraph Machine["Kubuntu desktop"]
        subgraph Bridge["whatsapp-bridge (Node.js/TS)"]
            Baileys["Baileys client\n(paired via QR)"]
        end

        Redis[("Redis\n(loopback only)")]

        subgraph Orchestrator["claude-orchestrator (Python)"]
            Dispatcher["Message dispatcher\n+ session/project state"]
            STT["faster-whisper\n(GPU, large-v3)"]
            MDConv["Markdown → WhatsApp\nformatter + splitter"]
        end

        CC["claude CLI\n(headless, --output-format stream-json)"]
        Inbox["Project inbox folders"]
    end

    Self <-->|WhatsApp Web protocol\noutbound only| Baileys
    Baileys <-->|pub/sub JSON| Redis
    Redis <-->|pub/sub JSON| Dispatcher
    Dispatcher --> STT
    Dispatcher --> MDConv
    Dispatcher -->|spawn / stream events| CC
    Dispatcher --> Inbox
    CC --> Inbox
```

Two independently-managed local processes (`whatsapp-bridge` and `claude-orchestrator`) communicate only through a local, loopback-only Redis instance. Neither process is reachable from outside the machine; the only outbound connection is the bridge's WebSocket to WhatsApp's own servers, which is why no public IP/VPS is required.

4. Components

4.1 `whatsapp-bridge` (Node.js / TypeScript)

- Uses **Baileys** (unofficial WhatsApp Web multi-device protocol library) to maintain a persistent connection to WhatsApp, paired once via QR code scan from the owner's phone.
- Baileys is Node/TypeScript-only, which is why this component exists in a different language from the rest of the system — its sole job is the WhatsApp wire protocol.
- Persists Baileys auth/session credentials to a local directory (e.g. `~/.local/share/whatsapp-bridge/auth/`) so re-pairing isn't needed after a restart.
- Filters all incoming events down to **only messages in the owner's own "Message Yourself" chat** (the thread where the account's own JID is both sender and recipient). Every other chat and group is ignored entirely at this layer — it is never even forwarded to Redis.
- **LID caveat:** WhatsApp's newer identity system means a self-chat message can arrive addressed under either the account's phone-number JID (`...@s.whatsapp.net`) or its LID JID (`...@lid`), depending on which client sent it — the phone app addresses it via LID, WhatsApp Desktop/Web via the phone-number JID. The bridge captures both identities at connection time (`socket.user.id` and `socket.user.lid`) and treats a match against *either* as the self-chat. A filter checking only one address would silently drop every message from the other client with no error.
- Downloads and decrypts any media (voice notes, images, documents) attached to a qualifying message using Baileys' media-download API, writing it to a shared staging directory (e.g. `/var/tmp/whatsapp-bridge/incoming/<message-id>/`).
- Publishes a normalized JSON event to Redis channel `whatsapp:incoming` for every qualifying message (see §6.1 for schema).
- Subscribes to Redis channel `whatsapp:outgoing` and sends any message published there back into the self-chat, splitting into multiple WhatsApp messages if the text exceeds ~4000 characters.
- Applies a defense-in-depth check as a second safety net even though self-chat scoping should already guarantee only the owner can write there: it confirms Baileys' own `message.key.fromMe` flag is `true`, rather than string-matching a sender JID (WhatsApp sends `participant: ""` — not absent — on self-chat messages, and the LID/phone-number ambiguity above makes JID string-matching unreliable here). The outgoing event to the orchestrator is stamped with the configured `allowed_sender_jid` from config, not a JID parsed off the message, so the orchestrator's own allow-list check (§9) stays simple.
- When a message is a WhatsApp reply/quote (`contextInfo.quotedMessage` present — a protobuf field nested per content type, not on the message envelope), extracts quoted text verbatim, or re-downloads a quoted attachment/voice note on the spot using the same media-download API (§5.9/§7.6). No lookup against anything the bridge saw earlier is involved — everything needed (including, for media, the same reference data used to re-download normal attachments) is already embedded in `contextInfo.quotedMessage` itself.
- Negotiates the current WhatsApp Web protocol version via Baileys' `fetchLatestBaileysVersion()` at startup, rather than the version bundled with the pinned Baileys release — the bundled version can lag behind what WhatsApp's servers actually expect, which otherwise surfaces as a handshake-level rejection indistinguishable from a real outage.
- Reconnects after a dropped connection use exponential backoff — 5s, 10s, 20s, ... capped at 5 minutes — rather than a fixed delay, so a server actively rejecting reconnects isn't hammered at a constant rate. The delay resets to 5s only once a connection has stayed open for a full 60 seconds; a flap that closes again

sooner doesn't count as recovery and keeps backing off from where it left off. After 10 failed attempts with no qualifying reset (~25 minutes total), the bridge logs an error and exits cleanly (exit code 0) rather than continuing to retry — `Restart=on-failure` in the systemd unit does not restart a clean exit, so the service is left stopped (`inactive (dead)`) until explicitly started again. A `loggedOut` disconnect is unaffected by any of this: it never reconnects at all and asks for re-pairing instead, same as before.

4.2 `claude-orchestrator` (Python)

- Subscribes to `whatsapp:incoming`, publishes to `whatsapp:outgoing`.
- Owns all business logic: command parsing, project/session state, transcription, Claude Code invocation, formatting, error handling, and cancellation.
- Maintains an in-memory (or lightweight local file/SQLite-backed) map of `project_name → { path, active_session_id, running_process }`.

4.3 Voice transcription

- `faster-whisper` running the `large-v3` model with the CUDA backend (CTranslate2), targeting the RTX 3060.
- Falls back to CPU (`medium` model) only if the GPU is unavailable at runtime (e.g. driver failure) — this is a resilience fallback, not the intended default path.
- Requires `ffmpeg` on the host for decoding WhatsApp's Opus/OGG voice notes before transcription.

4.4 Claude Code integration

- Invoked as the `claude` CLI in headless/print mode: `claude -p "<prompt>" --output-format stream-json --dangerously-skip-permissions` (flag names per the CLI version in use at implementation time; confirm exact flags against the installed Claude Code version).
- Runs with **working directory set to the active project's path**, and only that path — this is the sandboxing boundary since permissions are fully auto-approved.
- The CLI must be invoked under the same OS user/session that has completed the interactive `claude login` OAuth flow for the Pro subscription; no API key is configured or used anywhere in this system.
- Session continuity uses the CLI's own resume mechanism (`--continue` / `--resume <session-id>`) keyed per project, so conversation context persists across messages until explicitly reset.
- The orchestrator parses the `stream-json` event stream to (a) detect tool calls/results for progress reporting, and (b) capture the final response text.
- Every invocation carries a fixed `--append-system-prompt` telling Claude Code about the `whatsapp-outbox/` convention (§7.5). A resumed session (`--resume`) picks this up fresh on every turn, not just the one present at session start, so the convention stays in context for the session's whole lifetime, not only its first message.
- The subprocess's stdout is read with an explicit 32MB line buffer (`asyncio.create_subprocess_exec(..., limit=...)`), not the asyncio default of 64KiB — a single `stream-json` event line can exceed that default once file content (e.g. reading a large PDF referenced

from `whatsapp-inbox/`) ends up embedded in a tool-result line, which would otherwise crash the read with `LimitOverrunError` mid-task.

4.5 Redis

- Runs locally (standard OS package, its own systemd-managed service), bound to `127.0.0.1` only, no exposed network interface, no authentication required given it is unreachable from outside the loopback interface.
- Used purely as a pub/sub message bus between the two components — not as a durable queue or database.

4.6 Voice synthesis (TTS)

- `piper-tts` (ONNX-based, CPU by default — fast enough without a GPU, and avoids VRAM contention with §4.3's `large-v3` model on this machine's single GPU).
- Voice models download automatically on first use into `tts.models_dir`, mirroring `faster-whisper`'s own model-download behavior (§4.3) rather than requiring a separate manual setup step.
- `langdetect` picks which configured voice (English/Spanish) to use **per reply**, based on the language of that reply's own text — not the input message's language, and not a fixed config default (SPEC.md §7.4).
- Requires `ffmpeg` (already a prerequisite for §4.3) to convert Piper's WAV output to the Opus/OGG format WhatsApp voice notes require.
- Opt-in only, via `/voice on` (§7.4) — off by default, since synthesis is extra per-message work most sessions won't want.

5. Message Flows

5.1 Plain text message

1. Owner sends a text message in the "Message Yourself" WhatsApp chat.
2. `whatsapp-bridge` publishes it to `whatsapp:incoming`.
3. `claude-orchestrator` checks whether it's a command (§7) or a prompt for the active project.
4. If a prompt: checks whether that project already has a task running. - If busy: publish an immediate "⏳ still working on your previous request" reply and stop (see §5.5). - If free: publish an immediate "⏳ Working on it..." acknowledgment, then invoke `claude` with `--continue` / `--resume` for that project's session.
5. While the process runs, the orchestrator batches `stream-json` events and publishes a progress-update message roughly every 10 seconds (see §5.4).
6. On completion, the orchestrator formats the final response (§5.6) and publishes it to `whatsapp:outgoing`.
7. If voice replies are on (`/voice on`, §7.4), the orchestrator also synthesizes a voice note and publishes it as a second, separate `whatsapp:outgoing` message — a best-effort extra: a synthesis failure is logged but never blocks or retracts the text reply already sent in step 6. What gets synthesized is the short spoken-style summary Claude Code appends to its reply when voice is on (§7.4), falling back to the raw response text when no summary section is present.

8. After a successful (non-error, non-cancelled) completion, the orchestrator also scans the active project's `whatsapp-outbox/` folder and sends anything Claude Code placed there (§7.5, §5.9) — independent of whether voice replies are on.

5.2 Voice message

1. Owner sends a voice note in the self-chat.
2. `whatsapp-bridge` downloads/decrypts the audio to the staging directory and publishes an event referencing that file path (media, not text).
3. `claude-orchestrator` runs `faster-whisper` (GPU, `large-v3`) on the staged audio file to get a text transcript.
4. The transcript is treated exactly like a typed text message from that point on (step 3 onward in §5.1). The orchestrator should echo the transcript back to the owner as part of (or immediately before) the "Working on it..." acknowledgment, so the owner can catch transcription errors.

5.3 Attachment (file/image/document)

1. Owner sends a message with a file attached (with or without caption text) in the self-chat.
2. `whatsapp-bridge` downloads the file to the staging directory, publishes an event with the staged path and any caption text.
3. `claude-orchestrator` moves (not copies) the file into the active project's inbox folder — a fixed conventioned subfolder, e.g. `<project-path>/whatsapp-inbox/<timestamp>-<original-filename>`.
4. The orchestrator appends the saved relative path to the prompt text sent to Claude Code, e.g.: `User said: "<caption>" — attached file: whatsapp-inbox/2026-07-26T10-15-00-photo.png`.
5. Proceeds as a normal prompt (§5.1 step 4 onward).

5.4 Progress updates

- The orchestrator buffers all `stream-json` tool-call/tool-result events since the last flush.
- Every ~10 seconds while a task is still running, it sends one consolidated WhatsApp message summarizing what happened in that window (e.g. "🔧 Read 3 files, ran `pytest`, edited `app.py`").
- No update is sent if nothing new happened in that window, and no update is sent for tasks that finish faster than the first interval.

5.5 Busy / concurrency handling

- Only one active `claude` process per project at a time.
- A new prompt for a project that already has a running task gets an immediate reply (e.g. "⏳ Still working on your previous request for `<project>`. Send `/stop` to cancel it, or wait.") and is **not** queued — the owner must resend it after the task finishes or is stopped.
- Prompts for a *different* project than the one currently running are allowed to proceed independently (the "busy" restriction is per-project, not global).

5.6 Response formatting

- Claude Code's Markdown output is converted to WhatsApp-supported formatting on a best-effort basis: triple-backtick code blocks and inline code are preserved as WhatsApp monospace, `**bold**` → `*bold*`, headers are flattened to bold plain lines, tables are flattened to simple aligned text.
- Any response exceeding ~4000 characters is split across multiple sequential WhatsApp messages rather than truncated.

5.7 Error handling

- Any failure (non-zero exit, timeout, exception, denied action attempting to escape the sandboxed project directory) is caught by the orchestrator.
- The owner receives a short, human-readable summary (e.g. "❌ Claude Code failed after 2m — see log for detail: ").
- Full detail (stack trace, stderr, full stream-json event log) is written to a local log file, never sent verbatim over WhatsApp.

5.8 Cancellation

- Sending `/stop` while a task is running for the active project sends `SIGTERM` (escalating to `SIGKILL` after a grace period, e.g. 5s) to that project's active `claude` child process.
- The orchestrator confirms cancellation back to the owner (e.g. "🛑 Stopped.") and clears that project's "busy" state so a new prompt can be sent immediately.

5.9 Outgoing files from Claude Code (`whatsapp-outbox/`)

The reverse of §5.3 — Claude Code, not the owner, decides a file should leave the project (§7.5).

1. At some point during the task, Claude Code (told about the convention via the system prompt, §4.4) uses its own Write/Bash tools to save a file into `<project-path>/whatsapp-outbox/`.
2. Once the prompt completes successfully (§5.1 step 6-8), the orchestrator scans that folder's top level (not `sent/`).
3. Each file ≤ 64MB is moved into `whatsapp-outbox/sent/<filename>` and published to `whatsapp:outgoing` as its own message (mimetype guessed from the extension, caption set to the filename, `delete_after_send: false` since the orchestrator already relocated it to its permanent home).
4. Each file over 64MB is left in place (not moved, not sent) and reported back with a short error reply instead.

5.10 Replying to (quoting) an earlier message

Applies uniformly on top of §5.1-§5.3 and §7.3's `/c` — a reply can itself be text, voice, or an attachment, and can quote a message in either direction (something the owner sent, or something the orchestrator/Claude Code sent).

1. The bridge checks whether the incoming message carries `contextInfo.quotedMessage` (§4.1). If not, nothing changes from §5.1-§5.3.

2. If the quoted message was text, its text is included verbatim — no download, no lookup.
3. If the quoted message was an attachment or voice note, the bridge re-downloads it on the spot (a one-shot fetch, not a persisted mapping — §7.6 Q1) and stages it into `whatsapp-inbox/` exactly like a fresh attachment (§5.3), hash-suffixed the same way (§7.6).
4. If that download fails (e.g. the quoted media has since expired on WhatsApp's servers), the orchestrator falls back to a generic note naming what type of thing was quoted, without a file reference.
5. Either way, the resulting context is prepended to whatever the owner actually typed/said/attached, e.g. `Replying to attached file: whatsapp-inbox/2026-...-photo_a1b2c3d4.jpg\n\n<the new message>`, before proceeding as a normal prompt (§5.1 step 4 onward).

6. Redis Channel Schemas

6.1 `whatsapp:incoming` (published by bridge, consumed by orchestrator)

```
{
  "message_id": "string",
  "timestamp": "ISO-8601 string",
  "sender_jid": "string",
  "type": "text | voice | attachment",
  "text": "string | null",
  "media_path": "string | null",
  "media_mime_type": "string | null",
  "quoted_text": "string | null",
  "quoted_media_path": "string | null",
  "quoted_media_mime_type": "string | null",
  "quoted_media_type_label": "string | null"
}
```

The four `quoted_*` fields are populated regardless of `type` (§5.10) — a reply can itself be text, voice, or an attachment. `quoted_text` is set verbatim for quoted text; `quoted_media_path` / `quoted_media_mime_type` are set when a quoted attachment/voice note was successfully re-downloaded; `quoted_media_type_label` (e.g. `"image"`, `"voice message"`) is set whenever the quoted message was media, whether or not the download itself succeeded — it's the fallback description when `quoted_media_path` is absent.

6.2 `whatsapp:outgoing` (published by orchestrator, consumed by bridge)

```
{
  "text": "string | null",
  "in_reply_to": "message_id string | null",
  "media_path": "string | null",
```

```
  "media_mime_type": "string | null",
  "media_filename": "string | null",
  "caption": "string | null",
  "delete_after_send": "boolean"
}
```

Exactly one of `text` / `media_path` is set per message (§7.4/§7.5) — a voice-note or file reply is its own separate message on this channel, not a caption alongside the text reply for the same result. `caption` / `media_filename` are only meaningful alongside `media_path`. When `media_mime_type` starts with `audio/`, the bridge sends it as a WhatsApp `ptt` voice note (§7.4); otherwise as a document, using `media_filename` / `caption` (§7.5). `delete_after_send` tells the bridge whether to delete `media_path` once sent: `true` for TTS's own temp files (§4.6), `false` for `whatsapp-outbox/` files, which the orchestrator has already relocated to their permanent `sent/` home. Either way, this is the one case where the orchestrator hands the bridge a filesystem path to read directly — the reverse of §6.1's incoming media flow (whether that path lives under `bridge.outgoing_staging_dir` for TTS, or inside a project's own `whatsapp-outbox/sent/` for files, doesn't matter to the bridge; both processes run as the same OS user on the same machine).

7. Command Reference

All commands are plain text messages in the self-chat, starting with `/`.

Command	Effect
`/project <name-or-number>`	Switch the active project — any `config.yaml` project, or any project folder under `project_root` or one of its `project_containers` — by name or by its `/projects` list number — and start that project's Remote Control server (§7.7). Naming a container lists its projects instead. See §7.2.
`/projects`	List every available project (config.yaml plus every `project_root` project folder), numbered, with each container's projects grouped under it.
`/create <name>`	Create a new project folder under `project_root` (`<name>`) or under one of its containers (`<container>/<name>`) switch to it and start its Remote Control server — see §7.2, §7.7.
`/close [name-or-number]`	Stop a project's Remote Control server — the active project's when no argument is given. See §7.7.
`/new`	Reset the active project's Claude Code session — the next prompt starts a fresh conversation instead of continuing.
`/stop`	Cancel the currently running task for the active project, if any.
`/enable`	Allow messages to reach Claude Code (global, all projects).

Command	Effect
`/disable`	Stop messages from reaching Claude Code (global, all projects) — see §7.1.
`/status`	Report enabled/disabled state, active project, whether it's busy or idle, the active project's Claude Code session ID (or "none yet"), and the running Remote Control servers (or "none").
`/c <text>`	Send `<text>` to Claude Code verbatim as the prompt, bypassing the command table above — see §7.3.
`/voice on` | `/voice off`	Also send replies as a voice note, in the reply's own detected language (off by default) — see §7.4.
`/help`	List available commands.

Any message not starting with `/` is treated as a prompt (or, for voice/attachments, as described in §5.2/§5.3) for whichever project is currently active, **provided the enabled/disabled toggle (§7.1) is on**. There must always be a default active project on startup (first entry in the configured project list) so an owner who forgets to `/project` first still gets a sensible result.

7.1 Enabled/disabled toggle

The owner's everyday personal WhatsApp number is used for this integration, and the same "Message Yourself" self-chat also doubles as a personal notes/backup thread — so not every message typed there is meant for Claude Code. A global on/off toggle (not per-project — it gates the *channel*, not any particular project) controls this:

- While **enabled** (the default on first install), non-command messages behave exactly as described elsewhere in this document — text becomes a prompt, voice notes are transcribed then prompted, attachments are staged then prompted.
- While **disabled**, the orchestrator does nothing extra with non-command messages: no transcription, no attachment staging, no prompt to Claude Code, no reply at all. WhatsApp's own chat history is treated as sufficient "backup" — no separate archiving is implemented. This silence is specifically for non-command text/voice/attachments, since those could plausibly be personal notes.
- Commands (`/project`, `/projects`, `/close`, `/new`, `/stop`, `/enable`, `/disable`, `/status`, `/help`) always run regardless of the toggle state — otherwise there'd be no way to `/enable` again.
- Anything else starting with `/` — `/c` (§7.3) or an unrecognized command like `/usage` — is unambiguously an attempted command, never a personal note, so while disabled it gets an explicit "🔴 Claude Code is disabled. Send /enable first." reply instead of either running or going silent. This also means an unrecognized command's reply differs depending on toggle state: the disabled notice takes priority over "Unknown command" while disabled, since that's the more actionable fact.
- `/disable` does not cancel a task already running (`/stop` still does that separately); it only gates *new* prompts from being dispatched.

- The toggle state persists across orchestrator restarts (a small local JSON state file, independent of `config.yaml`), so a deliberate `/disable` survives a crash/reboot/service update instead of silently reverting.

7.2 Project discovery (`project_root`) and dynamic creation (`/create`)

Claude Code's working-directory confinement to a pre-configured project list (§9) is the only safety boundary once permissions are auto-approved, so letting a WhatsApp message target an arbitrary filesystem path is never allowed. `project_root` is the one deliberate, bounded exception — the entire subtree the owner already trusts enough to let Claude Code run against, so *nothing under it* needs a separate registration step to become switchable:

- **Every immediate, non-hidden subfolder of `project_root` is automatically an available project, unless it is a container** — discovered live (re-scanned on every `/project` / `/projects` call, not cached), merged with `config.yaml`'s static project list and deduplicated both by name (a static entry always wins a name collision) *and* by resolved path (a static entry named e.g. `wasap-grill-me` pointing at a folder literally called `wasap_grill_me` must not also appear a second time under that basename — same directory, two names, would otherwise become two independent sessions racing against one `cwd`). If `project_root` is unset, only the static list is available.
- **Containers (`project_containers`) hold projects instead of being one.** `config.yaml` lists them as paths relative to `project_root` (e.g. `c`, `c/car`). A container is scanned one level deep with the same rule as `project_root`: each non-hidden subfolder is a project, unless it is itself a listed container. Nested projects are named by their path relative to `project_root` (`c/whey`, `c/car/maintenance`), so a nested folder never collides with a top-level one of the same basename. A container is only reached through `project_root` or another listed container, so depth is set by what is listed, not by a separate limit. A listed container whose folder is missing is skipped, with a warning in the service log at startup. The container list is read once, at startup. A container itself is never selectable: `/project <container>` replies with that container's projects instead of switching.
- **No registration step, no persisted project list.** Switching to a project_root folder for the first time or `/create`-ing one simply adds it to the orchestrator's in-memory project registry for the rest of that process's lifetime (session continuity — the active Claude Code session id — included); nothing is written to disk beyond the folder itself. `project_root` is the single source of truth, so there's nothing else that needs to stay in sync with it.
- **`/project <name-or-number>`** accepts either a project name or its number from the current `/projects` listing. Numbers reflect a fresh scan each time, sorted alphabetically by path part (so a container's projects come right after the container) for a predictable (if not permanently stable) order — list, then act on it promptly.
- **`/projects` groups nested projects under their container.** Each container is an unnumbered 📁 `<container>` header, and its projects (and sub-containers) follow it, indented two spaces per level. The reply to `/project <container>` shows the same subtree with the same numbers.
- **`/projects` shows just the name for anything under `project_root`** — the path is redundant once every such project already shares that root. A static `config.yaml` entry whose path falls *outside*

`project_root` still shows its full path, since there's no shared root to omit it against.

- **`/create <name>`** still exists for the one thing discovery alone can't do: making a *new* empty folder, directly under `project_root` (`<name>`) or inside a listed container that exists (`<container>/<name>`, e.g. `c/car/newapp`). The new folder's own name must match `^[A-Za-z0-9_-]{1,100}$` — no slashes, no `..` — and its parent must be `project_root` or a known container, making path-traversal structurally impossible rather than relying on normalization logic to catch it. It's otherwise just `mkdir` + switch — no separate registration, since the folder is automatically a project the instant it exists. Fails safe on collision (name already available, from either source) rather than silently adopting or overwriting.

7.3 Escaping to Claude Code's own commands/skills (`/c`)

Claude Code CLI has its own slash-command vocabulary (built-ins like `/usage`, plus plugin-provided skills like `/mattpocock-skills:grill-me`) that is entirely separate from this integration's command table above. Since §7 already claims every message starting with `/` for the orchestrator's own commands, anything typed as e.g. `/usage` would otherwise be swallowed and rejected as "Unknown command" before ever reaching Claude Code.

`/c <text>` is the escape hatch: `<text>` is forwarded verbatim as the Claude Code prompt, exactly as if it had been sent without a leading `/c` (same busy-check, session-resume, and progress-update path as any other prompt — §5.4-§5.8). No special parsing of `<text>` is done on the orchestrator side; Claude Code CLI itself recognizes slash-command/skill syntax in the prompt it receives and dispatches accordingly, the same as the interactive CLI. Two Claude Code CLI limitations are inherent to this and not something `/c` can work around: some purely-interactive built-ins (e.g. `/help`) reply "isn't available in this environment" headless regardless of how the prompt reaches them; and plugin skills must be addressed by their fully-qualified `plugin:skill` name (`/mattpocock-skills:grill-me`) — a bare short name like `/grill-me` gets "Unknown command" from Claude Code CLI itself, not from the orchestrator.

`/c` is content bound for Claude, so — unlike the meta-commands in §7 — it is gated by the enabled/disabled toggle (§7.1): a `/c` sent while disabled replies with an explicit "Claude Code is disabled" notice rather than running (§7.1).

7.4 Voice replies (`/voice on` / `/voice off`)

Text and voice/attachment inputs already convert *to* text (§5.1-§5.3); this is the reverse direction — an optional voice note sent back *alongside* the text reply Claude Code already always produces.

- **Global toggle, off by default.** `/voice on` / `/voice off` is a second, independent toggle from §7.1's `claude_enabled` — same persisted-state-file pattern (`state.json`), same "always runs regardless of `claude_enabled`" treatment as the other meta-commands in §7, but defaults to off rather than on, since synthesis is extra per-message cost most sessions won't want.
- **Voice in addition to text, never instead of it.** Every reply while voice is on is delivered as both the existing text message(s) (§5.6) and one separate voice-note message (§6.2) — never a replacement, and progress updates (§5.4) stay text-only either way.

- **A short spoken summary, not a full readout.** While voice is on, Claude Code is instructed to end its reply with a `## 🔊 Audio Summary` heading followed by a spoken-style summary (~100 words or fewer) of the answer; the voice note speaks that summary rather than reading the full text reply aloud. The text reply itself is unaffected and always carries the complete answer. If Claude Code's reply has no summary section, the voice note speaks the full reply text instead.
- **Language is detected per reply, not configured or matched to the input.** Right before synthesis, `langdetect` classifies whichever text is actually being spoken (the summary when present, the full reply text otherwise) and picks the matching voice from `tts.voices` (English/Spanish, §4.6, §8) — asking in Spanish and getting an English-language reply back produces an English voice note, and vice versa. Any detected language with no configured voice falls back to `en`.
- **Markdown is stripped for speech only.** The text channel keeps full Markdown formatting (§5.6) unchanged; only the copy fed to Piper has formatting markers removed and fenced/inline code replaced with a short spoken placeholder ("(code block omitted)") — code read aloud character-by-character isn't meaningful.
- **Best-effort, never blocks the text reply.** A synthesis failure (model load error, `ffmpeg` failure, etc.) is logged and silently skipped — the text reply already sent is never held back or retracted waiting on voice.

7.5 Outgoing files from Claude Code (`whatsapp-outbox/`)

The reverse of §5.3's incoming-attachment convention — Claude Code itself, not the owner, decides a file should be delivered back (e.g. a generated PDF report), by placing it in a fixed folder inside the project it's already confined to.

- **Convention, not a tool.** Claude Code has no dedicated "send to WhatsApp" tool — it only has Read/Write/Bash inside the active project directory. It's made aware of the `whatsapp-outbox/` convention via a fixed `--append-system-prompt` string on every invocation (§4.4), not a per-project `CLAUDE.md` edit, so it works out of the box in any configured project.
- **Only that one folder is ever touched.** The orchestrator scans exactly `<project>/whatsapp-outbox/`'s top level after each successful prompt (§5.9) — nothing else Claude Code creates anywhere else in the project is ever sent.
- **Archived, not deleted, after sending.** Each sent file is moved into `whatsapp-outbox/sent/` as it's picked up — this is also what makes "already sent" durable across restarts with no separate state file: a file is pending if and only if it's still sitting in the top-level folder.
- **64MB cap, oversized files rejected with a clear reply.** A file over the cap is left in place (not moved to `sent/`, not sent) and reported back with a short error — it'll be picked up again on the next successful prompt once it's small enough or removed.
- **Any file type**, not restricted to PDF — mimetype is guessed from the file extension; unrecognized extensions fall back to `application/octet-stream`.
- **Auto-gitignored, git repos only.** The first time a project's `whatsapp-outbox/` is used, if that project already has a `.git` directory, `whatsapp-outbox/` is appended to its `.gitignore` (creating the file if needed) — one less manual step, without ever creating a stray `.gitignore` in a project that isn't a git repo to begin with.

- **Filenames are content-hash-suffixed** (e.g. `report_a3f9c2d1.pdf`) — see §7.6, which applies the same naming rule to `whatsapp-inbox/` too.

7.6 Replying to (quoting) an earlier message

WhatsApp's native reply/quote feature, distinct from forwarding — see §5.10 for the step-by-step flow. Two things worth calling out about the design, not just the mechanics:

- **No persistent tracking system.** A quoted attachment/voice note is simply re-downloaded fresh at reply-time (§5.10 step 3), the same one-shot media-download call already used for a normal attachment — not resolved through a WhatsApp-message-id → project-file-path index. No state file to maintain, no bridge→orchestrator acknowledgment channel, nothing that can fall out of sync across a restart.
- **Content-hash filenames, both directions.** Because Claude Code could plausibly generate the same filename (e.g. `report.pdf`) more than once across different tasks, and because replies mean the same underlying file can end up referenced from more than one place over time, both `whatsapp-inbox/` (§5.3) and `whatsapp-outbox/sent/` (§7.5) suffix the stored filename with the first 8 hex characters of the file's own sha256 (e.g. `report_a3f9c2d1.pdf`) — identical content always produces the identical suffix, and two different files never silently overwrite one another.

7.7 Remote Control servers (`/project`, `/create`, `/close`)

`/project` and `/create` also start a `claude remote-control` server in the project's folder, so the project shows up in the Code tab of the Claude mobile app as a full, live Claude Code session — skills, hooks, MCP servers and `CLAUDE.md` included. The `claude -p` path (§4.4) is unchanged and runs beside it.

- **One server per project, in its own detached tmux session.** The session name is `rc-` plus the project name with every character outside `[A-Za-z0-9_-]` replaced by `-` (`c/whey` → `rc-c-whey`); the same name is passed as `--name`, so it is what the app shows. Names, not `/projects` numbers, are used because the numbers shift as folders come and go (§7.2). From the laptop, `tmux attach -t rc-c-whey` opens the same server.
- **Started once.** If the project's tmux session is already live, `/project` reports "already live" and starts nothing. A session whose process has exited is removed and started again.
- **Sessions keep their mode.** A starting server brings back the sessions it served before (for about four hours after it stopped), and a brought-back session keeps the permission mode it was created with. `remote_control.permission_mode` applies to sessions the server creates. To get a session in the current mode, archive the old one in the app and start the server again after that window.
- **Command line:** `<claude_code.binary> remote-control --name rc-<name> --spawn <remote_control.spawn> --permission-mode <remote_control.permission_mode> --chrome|--no-chrome` (§8). The permission mode defaults to `bypassPermissions`, so sessions on the phone never ask for approval — the same trust the `claude -p` path runs with (`--dangerously-skip-permissions`, decision #3). `auto` still stops to ask about actions its safety check flags. `--chrome` or `--no-chrome` comes from `remote_control.chrome`, off by default: browser work in these sessions goes through the Playwright MCP plugin, not the owner's real, logged-in Chrome. The owner's own `/chrome` setting is not

consulted. `--continue` is not used: it resumes one earlier session in single-session mode, which exits when that session ends and accepts no new ones; the Code tab already lists earlier sessions.

- **The owner's own environment.** The server runs through the owner's login shell in interactive mode (`<shell> -lic`), so it and its sessions get the same environment as a terminal (`PATH`, Node version managers, `~/.bashrc`), not the orchestrator service's. The service's `LD_LIBRARY_PATH` (§10) is removed. The arguments reach the binary as `"$0" "$@"`, never parsed by the shell. Claude Code itself reads the owner's `~/.claude` settings, plugins, hooks and MCP servers, as for any session run by that user.
- **Folder trust.** The server has no one to answer Claude Code's folder-trust prompt, so before starting it the orchestrator sets `projects["<absolute path>"].hasTrustDialogAccepted = true` in `~/.claude.json` — the same flag the prompt writes. The file is written only when the flag is missing or false, through a temp file and rename. Only folders the dispatcher resolved from the project list (§7.2) are ever marked, never a path from message text.
- **Outside the service's lifetime.** The tmux server is started through `systemd-run --user --scope`, so it lives in its own scope unit and a restart of `claude-orchestrator.service` leaves every server running. Servers do not come back after a reboot or logout, and do not stop on their own when idle; `/close` stops one.
- **Reply.** The orchestrator waits up to 20 seconds for the `https://claude.ai/code…` URL the server prints once it is connected, then replies "🟢 rc- is live" with that URL. A server that exits first is reported with the last 15 lines of its output and removed. A server still without a URL after the wait is left running and reported as still starting. The project switch stands whatever the outcome.
- **Switching away does not close.** `/project` to another project leaves the earlier server running. `/status` lists every live `rc-*` session.

8. Configuration

A local config file (e.g. `~/.config/claude-orchestrator/config.yaml`) defines:

```
whatsapp:
  allowed_sender_jid: "<owner's WhatsApp JID>"

projects:
  - name: wasap-grill-me
    path: /home/centos/PycharmProjects/wasap_grill_me
  # additional projects as needed

# optional; parent directory /create may make new project folders under (§7.2)
project_root: /home/centos/PycharmProjects
# optional; folders under project_root that hold projects (§7.2)
project_containers:
  - c
  - c/car
```

```
claude_code:
  binary: claude
  # per-task hard timeout, e.g. 15m, after which the process is killed
  # and treated as a failure (§5.7)
  task_timeout_seconds: 900

# Remote Control server started by /project and /create (§7.7)
remote_control:
  # acceptEdits | auto | bypassPermissions | default | dontAsk | plan
  permission_mode: bypassPermissions
  # same-dir: every session works in the project folder itself
  # worktree: each new session gets its own git worktree (git repos only)
  spawn: same-dir
  # Claude in Chrome for spawned sessions; off means Playwright MCP only
  chrome: false

whisper:
  model: large-v3
  device: cuda
  fallback_device: cpu
  fallback_model: medium

# voice replies via /voice on (§7.4), off by default
tts:
  voices:
    en: en_US-ljspeech-high
    es: es_AR-daniela-high
  device: cpu  # intentional: avoids VRAM contention with whisper.device: cuda above
  models_dir: ~/.local/share/claude-orchestrator/piper-voices

redis:
  host: 127.0.0.1
  port: 6379

progress_updates:
  interval_seconds: 10

bridge:
  # shared: orchestrator writes synthesized voice-reply audio here, bridge
```

```
# reads/sends/deletes it (§6.2, §7.4) — the rest of this section is
# Node-bridge-only (auth_dir, staging_dir)
outgoing_staging_dir: /tmp/whatsapp-bridge/outgoing
```

9. Security Considerations

- **Blast radius containment:** Claude Code always runs with permissions auto-approved, so the *only* safety boundary is the working-directory confinement to configured project paths. The implementation must guarantee the `claude` process is never launched with a working directory outside the configured project list, and should not blindly trust paths embedded in message text for anything other than the attachment-inbox convention in §5.3. `project_root` (§7.2) is the one deliberate, bounded exception to "the project list is static" — both `/create` and switching to an already-existing folder are confined to that one pre-configured directory (regex-validated folder names, inside `project_root` or a listed container, for `/create`), so the boundary itself never moves; it just means the owner is trusting the *entire* `project_root` subtree up front, not each folder individually as it's first used.
- **Remote Control servers (§7.7) widen who drives a project, not where.** Each server runs in a project folder from the same project list, so the working-directory boundary above holds. What changes is the client: anyone signed in to the owner's Claude account can open a session on a live server from the Claude app or claude.ai. Setting `hasTrustDialogAccepted` skips the folder-trust prompt only for folders already inside that boundary; it is never set for a path taken from message text. The server's own permission mode (`remote_control.permission_mode`, §8) decides what a session may do without asking.
- **Self-chat scoping is the primary access control.** Only the account owner can write to their own "Message Yourself" thread, which is why this is a stronger boundary than sender-ID filtering alone. `message.key.fromMe` (§4.1) is retained as defense-in-depth in case of a future Baileys/WhatsApp behavior change around that chat.
- **No inbound network exposure.** Redis is loopback-only; the bridge makes only outbound connections to WhatsApp's servers. There is no listening port reachable from outside the machine.
- **Local secrets:** Baileys' auth/session directory contains credentials equivalent to a logged-in WhatsApp Web session — treat it like a credential store (filesystem permissions restricted to the owning user, excluded from any backups that leave the machine unencrypted).
- **Known trade-off, accepted by the owner:** using the everyday personal WhatsApp number (rather than a dedicated one) means the bridge process technically observes all account traffic at the protocol level, even though it acts only on the self-chat. This was a deliberate choice weighed against the convenience of not needing a second number.
- **`whatsapp-outbox/` (§7.5) is a new data-egress path, not a new sandbox boundary.** Claude Code already runs with permissions auto-approved inside its project directory (`--dangerously-skip-permissions`, first bullet above), so it could already read or transmit any file reachable from there before this feature existed — `whatsapp-outbox/` doesn't expand what Claude Code can access, it just gives an already-trusted process a direct, convenient channel to get a file off the machine over WhatsApp instead of e.g. pasting its contents into a text reply. The 64MB cap and the restriction to one specific folder (§7.5)

bound the *convenience* of that channel, not the underlying trust boundary, which was already accepted as part of running with permissions auto-approved at all (decision #3).

10. Deployment & Operations

1. **Prerequisites:** Node.js (current LTS), Python (≥3.11, matching existing `pyproject.toml` scaffold), `redis-server`, `ffmpeg` and `tmux` (via apt). GPU-accelerated `faster-whisper` needs CUDA/cuDNN libraries loadable at runtime; these come from the `nvidia-cublas-cu12` / `nvidia-cudnn-cu12` pip wheels (pulled in with the rest of the Python dependencies, no system-wide CUDA install needed), with the systemd unit's `LD_LIBRARY_PATH` pointing at those wheels' `lib/` directories inside the venv so they're found at process start.
2. **Disable automatic suspend/sleep** in Kubuntu power settings (screen lock/screensaver may remain enabled).
3. **Pairing:** run the bridge interactively once to scan the WhatsApp QR pairing code; confirm the auth directory persists across restarts.
4. **systemd units** (both `systemctl --user`, so they start on login and stop on logout, per §9 in the interview): - `whatsapp-bridge.service` — runs the Node process. - `claude-orchestrator.service` — runs the Python process, `Requires=` / `After=` the bridge unit. - Both set `Restart=on-failure`. - `loginctl enable-linger <user>` should specifically **not** be used, since the design intentionally ties the service lifetime to an active login session rather than running unattended without any session.
5. **Logging:** each service logs to the systemd journal (`journalctl --user -u <unit>`) plus an application-level log file with rotation (e.g. Python `logging` with a `RotatingFileHandler`) for the detailed error traces referenced in §5.7.

11. Out of Scope (for this iteration)

- Official WhatsApp Business Cloud API integration.
- Group chat support.
- Multi-user support beyond the single owner allow-list.
- A dedicated/second WhatsApp number for the bot.
- Any inbound network exposure, tunneling, or remote-wake capability for the host machine.

12. Decision Log

#	Decision	Choice
1	Messaging channel	Unofficial WhatsApp automation (Baileys)
2	Access control	Sender allow-list
3	Claude Code permissions	Auto-approved, confined to project folders

#	Decision	Choice
4	Project scope	Multiple projects, switchable via chat command
5	Session continuity	Persistent per project until `/new`
6	Voice transcription engine	Local, GPU-accelerated (`faster-whisper`)
7	Attachment handling	Fixed inbox folder inside active project
8	Long-task feedback	Immediate ack + streamed progress
9	Progress throttling	Time-based batching, ~10s
10	Process persistence	Login-scoped `systemd --user` service; auto-suspend disabled
11	Error handling	Sanitized WhatsApp summary + full local log
12	Response formatting	Markdown→WhatsApp conversion + auto-split
13	Language stack	Python orchestrator + Node/TS Baileys sidecar
14	Inter-process communication	Local Redis pub/sub
15	WhatsApp number	Owner's everyday personal number
16	Message scoping	"Message Yourself" self-chat only
17	Cancellation	`/stop` command (SIGTERM/SIGKILL)
18	Concurrency	Reject with notice; no queueing, no parallel runs
19	Whisper model	`large-v3`, GPU-accelerated
20	Claude routing control	Stateful `/enable` / `/disable` / `/status` toggle (not a per-message marker)
21	Disabled-message handling	Do nothing extra — WhatsApp's own history is the backup, no separate archiving
22	Toggle scope	Global (all projects), not per-project
23	Toggle persistence	Persisted to disk, survives restarts; fresh install defaults to enabled
24	`/create` UX	One combined command (create + switch), not two steps
25	`/create` naming rule	Regex-validated folder name, placed directly under `project_root` or inside a listed container — never an arbitrary path

#	Decision	Choice
26	`/create` collision handling	Reject on any existing name — from `config.yaml`, a `project_root` folder, or an on-disk folder — never silently adopt or overwrite
27	`/create` command naming	`/create`, not `/newproject` — avoids clashing with `/new` and `/project`
28	Reaching Claude Code's own commands/skills	Dedicated escape command `/c <text>`, not a blanket fallthrough of unrecognized `/word` text — keeps the typo-safety of "Unknown command" errors for the orchestrator's own commands
29	`/c` toggle interaction	Gated by `/enable` / `/disable`, not always-on like the meta-commands — it delivers content to Claude, which is exactly what `/disable` suppresses
30	Disabled-state reply for slash-commands	`/c` and unrecognized commands (e.g. `/usage`) get an explicit "Claude Code is disabled" reply while disabled, not silence and not "Unknown command" — a leading `/` is never a personal note, so silence (§21) doesn't apply, and the disabled state is more actionable than a bare "unknown command" error
31	Voice-reply trigger	Global `/voice on` / `/voice off` toggle, independent of input modality — not "mirror the input" (voice-in → voice-out), not always-on, not a per-message opt-in
32	Voice-reply mode	Voice in addition to text, never instead of it — avoids losing the searchable/copyable text record
33	Voice command naming	`/voice on` | `/voice off`, matching `/project <name>`'s single-command-plus-argument shape rather than `/enable` / `/disable`'s two-distinct-verbs shape
34	TTS engine	Local `piper-tts`, CPU — fast enough without a GPU, avoiding VRAM contention with `faster-whisper`'s `large-v3` on this machine's single GPU
35	Voice language selection	Detected per-reply via `langdetect`, from the reply's own text — not the input message's language, and not a static config default
36	Voice model download	Automatic on first use, matching `faster-whisper`'s existing behavior — not a separate manual setup step
37	TTS text preprocessing	Markdown stripped and code blocks omitted for the spoken copy only; the text-channel reply keeps full Markdown unchanged
38	Outgoing-file trigger	Filesystem convention (`whatsapp-outbox/`), not a marker parsed out of Claude Code's free-form reply text — a folder Claude Code already has Write access to is more reliable than requiring exact parseable syntax in prose

#	Decision	Choice
39	Making Claude Code aware of the outbox convention	Fixed `--append-system-prompt` on every invocation, not a per-project `CLAUDE.md` edit — works automatically in any configured project, including resumed sessions
40	Outbox scope	Only the top level of `whatsapp-outbox/` itself is ever scanned/sent — nothing else Claude Code creates elsewhere in the project is touched
41	Sent-file handling	Moved into `whatsapp-outbox/sent/` (kept, gitignored), not deleted — durable "already sent" tracking with no separate state file, and no loss if a send silently fails
42	Outbox gitignore handling	Automatic, but only for projects that are already git repos — one less manual step without ever surprising a non-git folder with a new file
43	Outbox file type/size limits	Any file type (not PDF-only), capped at 64MB with a clear rejection reply above the cap — no type allow-list to maintain, but no unbounded/silent large-file attempts either
44	Outbox document caption	Filename shown as the WhatsApp caption, not sent bare
45	Reply/quote scope	Text and attachments/voice notes, in either direction — not restricted to only the owner's own earlier messages
46	Quoted-media handling	Re-downloaded fresh at reply-time (one-shot, via the same media-download call as a normal attachment) rather than resolved through a persisted message-id→file-path index — keeps quoting stateless, with nothing to fall out of sync across a restart. Quoted voice notes are handled the same way, with no separate re-transcription step
47	Content-hash filenames	Both `whatsapp-inbox/` and `whatsapp-outbox/sent/` suffix filenames with the first 8 hex chars of the file's sha256 — prevents same-named files (e.g. a regenerated report, or a file referenced by more than one reply over time) from silently overwriting one another
48	Project discovery model	`project_root` itself is the source of truth, live-scanned on every call — not a persisted registry that has to be kept in sync with it
49	Numbered project switching	`/project <name-or-number>` accepts either, resolved against a fresh `/projects`-style scan each time — not a separate command, not a stable/persisted numbering scheme
50	Project adoption scope	First switch to a `project_root` folder adopts it in-memory only for that process's lifetime (session continuity preserved) — nothing persisted to disk,

#	Decision	Choice
		since `project_root` itself is already the durable record
51	Discovery dedup key	By resolved path, not just by name — a static `config.yaml` project can have a name that doesn't match its own folder's basename, so path-based dedup prevents the same directory from ever appearing under two different names, which would otherwise let two independent sessions run against one directory at once
52	`/projects` path display	Name only for anything under `project_root` — the path is redundant once every such project shares that root. A static `config.yaml` entry outside `project_root` still shows its path
53	Reconnect rate limiting	Exponential backoff (5s→5min cap), reset only after 60s of stable connection, give up after 10 failed attempts (~25 min) — not a fixed retry delay, and not unlimited retries at the capped rate
54	Give-up behavior	Clean `process.exit(0)` so `Restart=on-failure` leaves the service stopped, rather than an in-process idle state or a failing exit that would just restart the whole cycle
55	WhatsApp protocol version	Fetched via `fetchLatestBaileysVersion()` at startup, not the version bundled with the pinned Baileys release — avoids a stale protocol version causing handshake rejections that look identical to a real outage
56	Voice-summary selection	A `## 🔊 Audio Summary` heading parsed out of Claude Code's free-form reply text selects the short spoken-style summary to speak (§7.4) — unlike decisions #20 and #38's preference against markers-in-free-text, a missing or malformed heading here degrades gracefully to speaking the full reply (already the behavior when no summary is present) rather than silently breaking an action, which is what motivated those two rules
57	Nested project discovery	Containers are listed by hand in `project_containers`, not guessed from marker files — a folder's contents don't reliably say whether it is a project or a folder of projects
58	Nested project naming	Path relative to `project_root` (`c/car/maintenance`), not the basename — basenames collide across containers
59	Container selection	A container is shown as an unnumbered header and is never selectable; `/project <container>` lists its projects
60	Remote Control process host	A detached tmux session per project, started through `systemd-run --user --scope` — tmux lets the owner attach from the laptop, and the scope keeps servers alive across orchestrator restarts

#	Decision	Choice
61	Remote Control naming	`rc-` + the project name with separators replaced by `-`, used for both the tmux session and `--name` — `/projects` numbers shift, names do not
62	Remote Control trigger	`/project` and `/create` start the server; `/close` stops it, separate from `/stop`, which keeps cancelling the `claude -p` task
63	Remote Control folder trust	Set `hasTrustDialogAccepted` in `~/.claude.json` before launch — the server has no one to answer the trust prompt, and only project-list folders are ever marked
64	Remote Control resume	No `--continue` — it runs a single session that exits when done; a fresh server accepts many sessions and the app already lists earlier ones
65	Remote Control lifetime	On demand only: no restart after reboot, no idle timeout, switching away keeps the server; `/status` lists live servers
66	Remote Control environment	The server starts through the owner's interactive login shell so phone sessions behave like terminal sessions, with Claude in Chrome off so browser work goes through Playwright MCP rather than the owner's logged-in Chrome; permission mode defaults to `bypassPermissions` because `project_root` is meant to run without approval prompts, and `auto` still asks about flagged actions